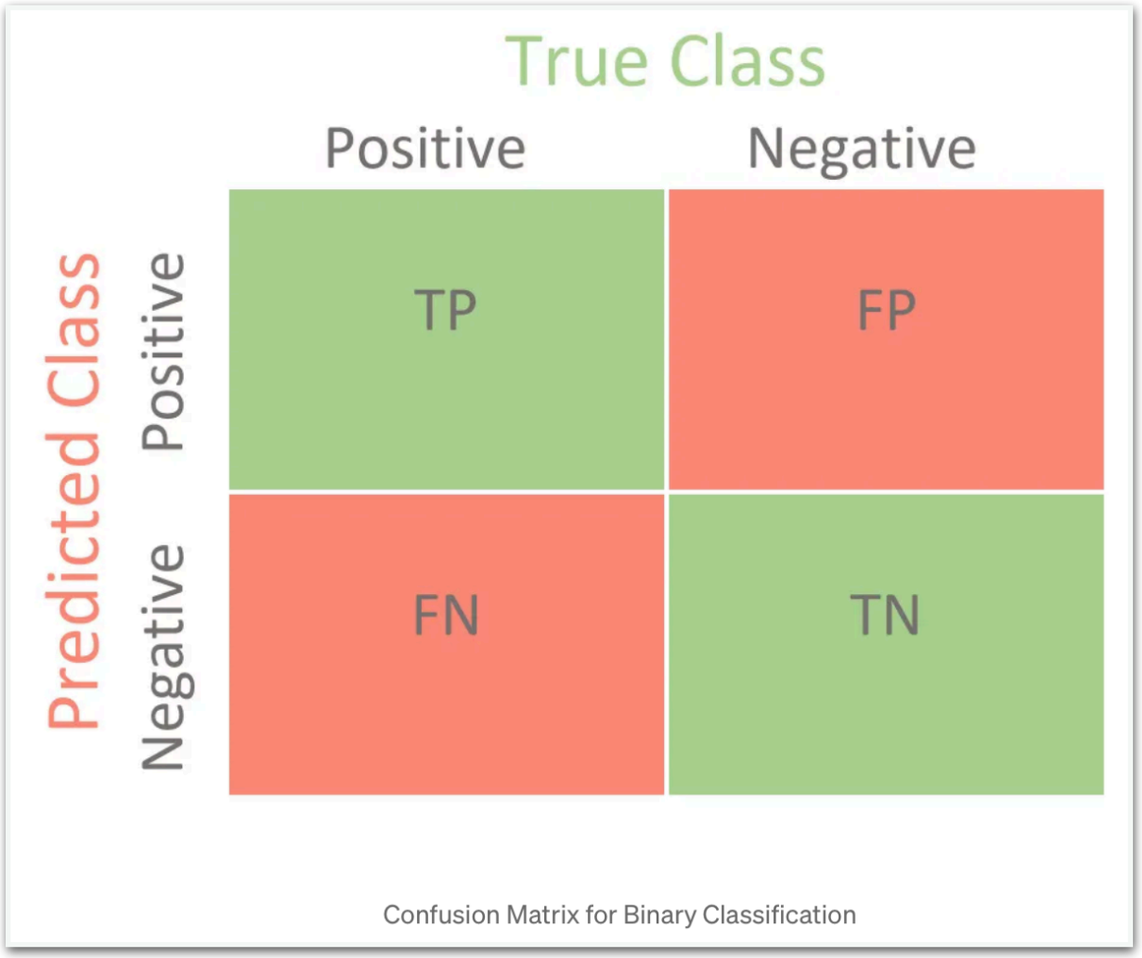
Confusion matrix



Confusion Matrix for Binary Classification

Key components: It breaks down predictions into four categories at a specific threshold:

- **True Positives (TP)**: Correctly predicted positive class.
- **True Negatives (TN)**: Correctly predicted negative class.
- **False Positives (FP)**: Incorrectly predicted as positive (Type I error).
- **False Negatives (FN)**: Incorrectly predicted as negative (Type II error).

TPR, FRN, FPR, TNR

- True positive rate (TPR) = sensitivity, recall, hit rate

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{P}}$$

- False negative rate (FNR) = missing rate

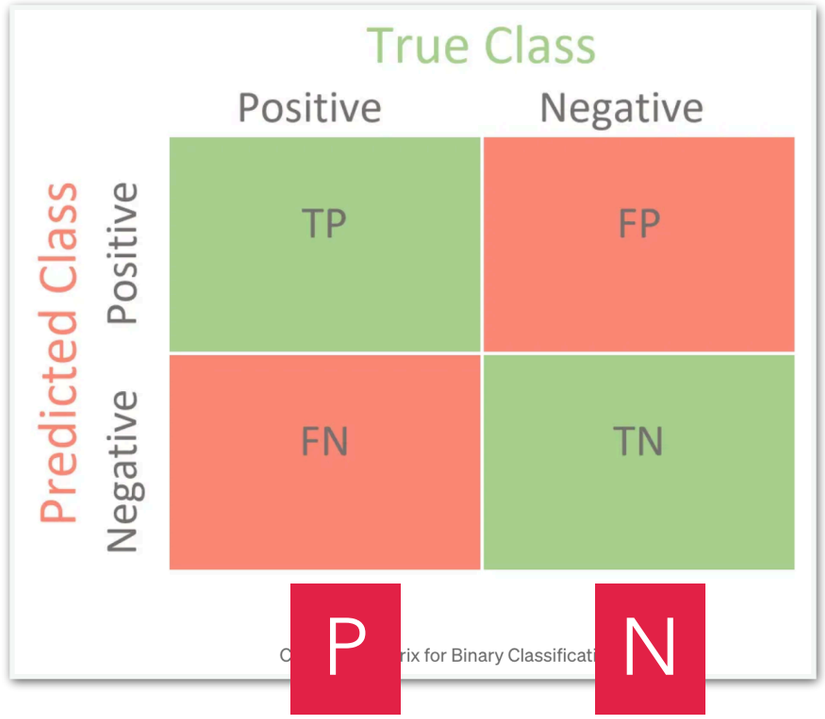
$$\mathrm{FNR} = \frac{\mathrm{FN}}{\mathrm{P}}$$

- False positive rate (FPR) = fall-out

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{N}}$$

- True negative rate (TNR) = specificity

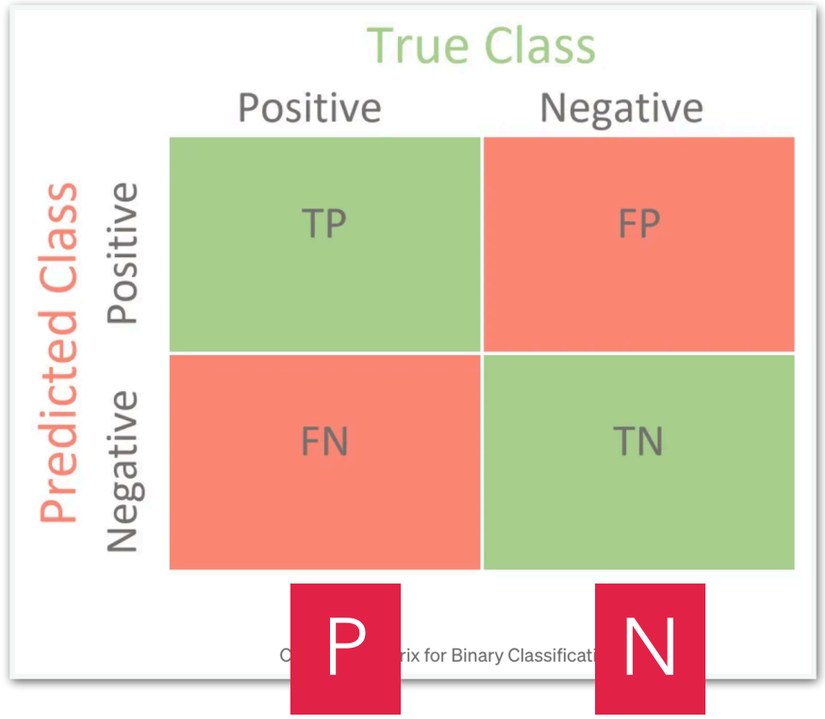
$$\mathrm{TNR} = \frac{\mathrm{TN}}{\mathrm{N}}$$



Useful measures

accuracy (ACC)

$$\mathrm{ACC} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{P} + \mathrm{N}} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$$



- Precision

$$\text{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

F1 score

is the harmonic mean of precision and sensitivity:

$$\mathrm{F}_1 = 2 \times \frac{\mathrm{PPV} \times \mathrm{TPR}}{\mathrm{PPV} + \mathrm{TPR}} = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

See Page A: Q1, Q2 👈

Thresold = 0.5

Predicted Probability	True Label	Predicted Label
0.9	1	1
0.8	1	1
0.7	0	1
0.6	1	1
0.5	0	1
0.4	1	0
0.3	0	0
0.2	0	0

	Predicted Positive (1)	Predicted Negative (0)
Actual Positive (1)	TP: 3	FN: 1
Actual Negative (0)	FP: 2	TN: 2

Threshold = 0.8

Predicted Probability	True Label	Predicted Label
0.9	1	1
0.8	1	1
0.7	0	0
0.6	1	0
0.5	0	0
0.4	1	0
0.3	0	0
0.2	0	0

	Predicted Positive (1)	Predicted Negative (0)
Actual Positive (1)	TP: 2	FN: 2
Actual Negative (0)	FP: 0	TN: 4

$$\hat{P}(Y = 1 \mid X), \textbf{given a threshold} \Rightarrow \hat{Y}$$

Depending on threshold

- TPR, FNR, FPR, TNR
- Accuracy
- Precision
- F1-score

Threshold free:
ROC curve
AUC

See Page A: Q3 👈

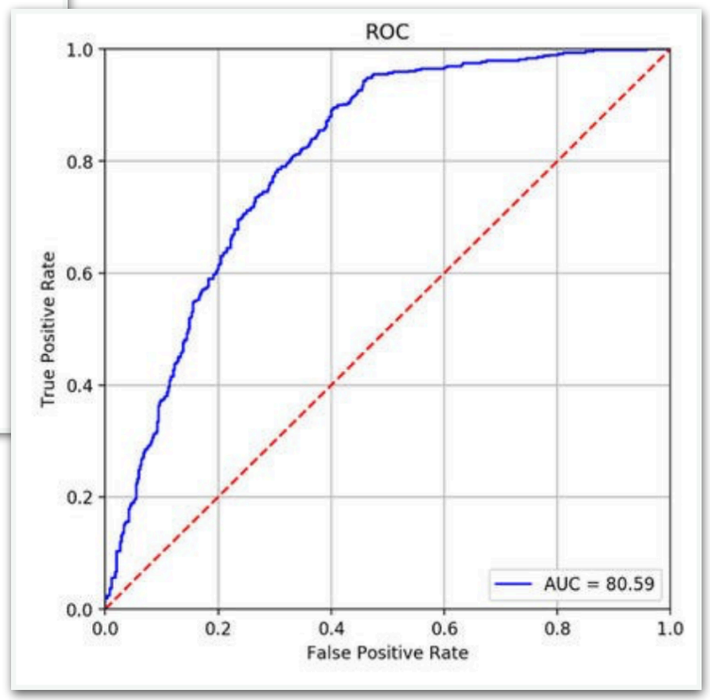
ROC curve

Google AI review

- **What it is:** A graph that plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.
- **Axes:**
  - **Y-axis**: True Positive Rate (Sensitivity or Recall), calculated as $TP/(TP+FN)$.
  - **X-axis**: False Positive Rate, calculated as $FP/(FP+TN)$.
- **Key metric:** The Area Under the Curve (AUC).
  - An AUC of 1.0 is a perfect classifier.
  - An AUC of 0.5 is a random classifier.
  - The higher the AUC, the better the model's ability to distinguish between classes.
- **Best for**: Evaluating a classifier's performance across all possible thresholds to understand its overall ability to discriminate between classes.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{P}}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{\text{FP}}{\text{N}}$$



See Page B: Q1, Q2, Q3 👈

Softmax

- Softmax turns K scores into K probabilities:

$$p_k(x_i) = P(Y_i = k \mid x_i) = \frac{\exp(\beta_k^\top x_i)}{\sum_{\ell=1}^{K} \exp(\beta_\ell^\top x_i)}, \quad k = 1, \ldots, K$$

- Each class has its own score, and every p is positive; together they sum to 1.

Logistic Regression with multiple class

- Suppose we have K classes.
- Each class uses parameter vector $\beta_k = (\beta_{0k}, \beta_{1k}, \beta_{2k}, \ldots, \beta_{pk})^\top$
- The sigmoid function $\sigma(t) = \dfrac{1}{1+\exp(-t)} = \dfrac{\exp(t)}{1+\exp(t)}$
- $\exp(\cdot)$ transfers a number to positive number
- $P(Y=k) \propto \exp(\beta_k^\top X) = \exp\left(\beta_{0k} + \beta_{1k}X_1 + \beta_{2k}X_2 + \cdots + \beta_{pk}X_p\right)$

Multinomial logistic regression as a GLM

- $y \mid x \sim \mathit{Multinomial}\,(1;\ p_1(x;\beta), \ldots, p_K(x;\beta))$
  - Error distribution: Multinomial distribution
  - Mean $\mu_k(x;\beta) = p_k(x;\beta) = \dfrac{e^{\beta_k^\top x}}{\sum_{\ell=1}^{K} e^{\beta_\ell^\top x}}$
  - $f(y_i \mid x_i;\beta) = \prod_{k=1}^{K} p_k(x_i;\beta)^{y_{ik}}$, where $y_{ik} = 1$ if $y_i = k$, else 0

Multinomial logistic regression as a GLM

- Likelihood:

$$L(\beta) = \prod_{i=1}^{n} f(y_i \mid x_i; \beta) = \prod_{i=1}^{n} \prod_{k=1}^{K} p_k(x_i; \beta)^{y_{ik}}$$

- Log-Likelihood:

$$\log L(\beta) = \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log p_k(x_i; \beta)$$

- Cross-entropy loss

$$\mathrm{CE}(\beta) = -\sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log p_k(x_i; \beta)$$

- $\implies \hat{\beta} = \arg\min_{\beta} \ \mathrm{CE}(\beta) = \arg\max_{\beta} \ L(\beta)$

Logistic regression with more than two classes

So far we have discussed logistic regression with two classes. It is easily generalized to more than two classes. One version (used in the R package `glmnet`) has the symmetric form

$$\Pr(Y = k|X) = \frac{e^{\beta_{0k}+\beta_{1k}X_1+\ldots+\beta_{pk}X_p}}{\sum_{\ell=1}^{K} e^{\beta_{0\ell}+\beta_{1\ell}X_1+\ldots+\beta_{p\ell}X_p}}$$

$\beta_k = (\beta_{0k}, \beta_{1k}, \ldots, \beta_{pk})^\top$

Here there is a linear function for *each* class.
(The *mathier* students will recognize that some cancellation is possible, and only $K - 1$ linear functions are needed as in 2-class logistic regression.)

Multiclass logistic regression is also referred to as *multinomial regression*.

Multinomial: the baseline coding (§4.3.5)

- Pick one class as the baseline, say class K. Only $K-1$ linear functions are needed.

- For $k = 1, \ldots, K-1$:

$$P(Y = k \mid X) = \frac{\exp(\beta_{0k} + \beta_{1k}X_1 + \cdots + \beta_{pk}X_p)}{1 + \sum_{\ell=1}^{K-1} \exp(\beta_{0\ell} + \beta_{1\ell}X_1 + \cdots + \beta_{p\ell}X_p)}$$

- For the baseline:

$$P(Y = K \mid X) = \frac{1}{1 + \sum_{\ell=1}^{K-1} \exp(\beta_{0\ell} + \beta_{1\ell}X_1 + \cdots + \beta_{p\ell}X_p)}$$

- Take the ratio:

$$\log \frac{P(Y = k \mid X)}{P(Y = K \mid X)} = \beta_{0k} + \beta_{1k}X_1 + \cdots + \beta_{pk}X_p$$

- This is a LOG-ODDS against the baseline, exactly as in two-class logistic regression. $K = 2$ gives back ordinary logistic regression.

Two codings, one model

- Softmax (page 35): K linear functions, symmetric, no baseline. Used by glmnet and sklearn.
- Baseline: K – 1 linear functions; the baseline class has all coefficients equal to zero.
- Same model. Subtract one vector c from every β_k: numerator and denominator are both multiplied by $\exp(-c^\top X)$, so every probability is unchanged.
- Mapping: β(baseline, class k) = β(softmax, class k) – β(softmax, class K).
- So the coefficient tables differ, but fitted probabilities and predictions agree.
- Interpret a multinomial coefficient only relative to the baseline that was chosen.